

全国大学生信息安全作品赛参赛作品

APRA 自适应渐进鲁棒聚合防御方法

Adaptive Progressive Robust Aggregation for Backdoor-Resilient Federated Learning

作品名称	APRA 自适应渐进鲁棒聚合防御方法
作品类别	信息安全作品
队伍成员	黄康 黄智辉 何明迅
所属学校	_____
指导教师	_____

源代码仓库: <https://github.com/GreenInsect/APRA>

摘要

联邦学习允许多个参与方在不共享原始数据的前提下协同训练机器学习模型，在金融、医疗、自动驾驶等隐私敏感领域具有广泛应用前景。然而，联邦学习的分布式特性使其面临严重的后门攻击威胁——攻击者可通过控制部分客户端向全局模型植入后门，使模型在带触发器的输入上输出攻击者指定类别，而主任务准确率几乎不受影响。

本作品提出 APRA (Adaptive Progressive Robust Aggregation, 自适应渐进鲁棒聚合) ——一种面向联邦学习后门防御的多阶段鲁棒聚合方法。APRA 的核心思想是将恶意客户端检测问题转化为多维度证据融合与渐进式降权问题：首先通过多维特征提取（参数更新展平、L2 范数、NBD/NDIF 行为探针）将客户端更新转化为可比较的数值特征；随后依次通过自适应 MAD 预过滤、层次聚类与可信簇选择、信任加权与自适应裁剪三个阶段，逐步缩小可疑客户端的聚合贡献。

在 CIFAR-10 和 CIFAR-100 两个数据集上使用 ResNet18 模型的 500 轮联邦训练实验中，APRA 在 A3FL、DOBA、ModelReplace、Neurotoxin、ReBA 五类主流后门攻击下均表现出色：CIFAR-10 上平均主任务准确率 92.13%，平均 ASR 仅 14.10%；CIFAR-100 上平均准确率 65.49%，平均 ASR 仅 1.48%。APRA 在两个数据集上均显著优于 FedAvg、Clip、DeepSight、FoolsGold 和 RFLBAT 等基线方法，尤其在 CIFAR-100 上将平均 ASR 降低了超过 36 个百分点。同时，APRA 内置的全过程审计追踪机制能够记录每轮每个客户端的筛选状态、信任权重和裁剪因子，为防御行为提供可解释性。同时，APRA 内置的全过程审计追踪机制能够记录每轮每个客户端的筛选状态、信任权重和裁剪因子，为防御行为提供可解释性。

目录

摘要	2
1. 作品概述	5
1.1. 背景介绍	5
1.1.1. 联邦学习的兴起与安全挑战	5
1.1.2. 后门攻击的严重性	5
1.2. 应用前景	5
1.2.1. 维护网络空间安全	5
1.2.2. 保障金融数据隐私与模型安全	5
1.2.3. 支撑医疗健康领域的安全协同	5
1.2.4. 赋能自动驾驶与物联网安全	6
1.3. 相关工作	6
1.3.1. 联邦学习后门攻击方法	6
1.3.2. 联邦学习后门防御方法	6
1.4. 作品概述与创新点	6
2. 作品设计	7
2.1. 问题分析	7
2.1.1. 现有防御方法的局限性	7
2.2. 设计思想	8
2.3. 系统架构	8
3. 作品实现	9
3.1. 多维特征提取	9
3.1.1. 参数特征提取	9
3.1.2. NBD/NDIF 行为探针	10
3.1.3. PCA 降维	11
3.2. 自适应 MAD 预过滤	12
3.2.1. MAD 鲁棒统计	12
3.2.2. 自适应阈值衰减	12
3.2.3. 安全保留机制	12
3.3. 层次聚类与可信簇选择	13
3.3.1. 簇数自动选择	13
3.3.2. 可信簇选择	13
3.4. 信任加权与自适应裁剪	14
3.4.1. 信任权重计算	14
3.4.2. 自适应裁剪	14
3.5. 过程追踪与可审计性	15
3.5.1. 客户端级追踪	16
3.5.2. 轮级摘要	16
4. 实验与分析	16
4.1. 实验设置	16
4.2. 主任务准确率	17

- 4.3. 攻击成功率 18
- 4.4. APRA 过程筛选分析 19
- 4.5. 整体对比总结 20
- 4.6. 消融分析 20
- 5. 创新性分析 21
 - 5.1. 方法创新 21
 - 5.1.1. 多阶段证据融合机制 21
 - 5.1.2. 软降权策略替代硬过滤 21
 - 5.1.3. 自适应阈值调度 22
 - 5.2. 工程创新 22
 - 5.2.1. 统一实验对比框架 22
 - 5.2.2. 全过程可审计追踪 22
 - 5.2.3. Non-IID 场景下的鲁棒性设计 22
- 6. 总结与展望 22
 - 6.1. 作品总结 22
 - 6.2. 未来展望 23
- 7. 参考文献 24

作品概述

背景介绍

联邦学习的兴起与安全挑战

联邦学习（Federated Learning, FL）作为一种分布式机器学习范式，允许多个客户端在服务端协调下共同训练模型，而无需将本地原始数据上传至中心服务器。这一特性使其天然契合日益严格的数据隐私法规（如 GDPR、《个人信息保护法》等），并在智能手机输入法优化、医疗影像分析、金融风控等场景中得到了广泛部署。

然而，联邦学习的分布式架构也打开了新的攻击面。由于服务端无法直接审查客户端的本地数据和训练过程，恶意客户端可以提交精心构造的模型更新，在不显著影响全局模型主任务表现的前提下，向模型中植入后门行为。一个典型的后门攻击场景是：攻击者在本地训练时将特定触发器（如像素模式、水印等）与目标类别关联，使得全局模型在遇到带触发器的输入时稳定输出攻击者指定的错误类别，而在干净样本上表现正常。这种隐蔽性使得后门攻击成为联邦学习面临的最严峻安全威胁之一。

后门攻击的严重性

后门攻击的危害跨越多个关键应用领域。在自动驾驶场景中，被植入后门的模型可能在识别到特定路标图案时将停车标志误分类为限速标志；在医疗诊断系统中，带有特定纹理的医学影像可能被系统地误诊为目标疾病；在金融风控中，带有特定交易特征的欺诈行为可能被模型系统性放过。这些攻击不仅威胁系统安全，更可能造成人身伤害和重大经济损失。

从技术角度看，联邦学习后门攻击具有三个令人担忧的特征：第一，攻击者只需控制少量客户端即可实现高成功率攻击；第二，攻击可以与正常训练同时进行，服务端难以从准确率指标上察觉异常；第三，持久性攻击（如 Neurotoxin）在攻击停止后仍能使后门长期残留，增加了防御和清除的难度。

应用前景

本作品研究的联邦学习后门防御技术具有广泛的应用前景，主要体现在以下方面：

维护网络空间安全

随着联邦学习在关键信息基础设施中的部署日益增多，后门攻击已成为网络安全防御的新战场。有效的后门防御方法能够保障分布式 AI 系统的模型完整性，防止攻击者通过控制边缘节点向核心模型注入恶意行为，从而维护国家级网络空间安全边界。

保障金融数据隐私与模型安全

金融行业是联邦学习最早也是最重要的应用领域之一。银行、保险等机构需要在跨机构联合建模（如联合风控、反洗钱）中保护客户隐私数据。本作品的方法可以在不访问原始数据的前提下，检测并抑制来自恶意参与方的模型投毒行为，保障联合建模结果的可靠性。

支撑医疗健康领域的安全协同

医疗数据具有极高的隐私敏感性和跨机构协同需求。多家医院联合训练疾病诊断模型时，必须防范恶意参与方植入可能危及患者安全的后门。APRA 的细粒度过程追踪能力在此场景中尤为重要——它可以追溯每一轮训练中每个参与方的行为，为医疗 AI 的安全合规提供审计依据。

赋能自动驾驶与物联网安全

自动驾驶和 IoT 设备通常作为联邦学习的边缘客户端参与训练，这些终端设备更容易被物理攻破或远程控制。APRA 通过多维特征的综合判断，不依赖单一假设来识别恶意更新，对终端安全防护薄弱的边缘场景具有更好的鲁棒性。

相关工作

联邦学习后门攻击方法

近年来，联邦学习后门攻击的研究发展迅速，攻击手段日趋复杂和隐蔽：

- 模型替换攻击 (Model Replacement)：攻击者在本地训练中将后门目标嵌入模型，然后通过放大恶意更新权重直接替换全局模型。此类攻击更新幅度大、方向集中，容易被基于范数的防御检测。
- A3FL (Adversarially Adaptive Backdoor Attacks to Federated Learning)：A3FL 在训练过程中自适应地调整攻击策略，通过对抗性优化使后门更新在参数空间中更接近正常更新分布，从而绕过基于更新幅度或方向的单一防御规则。
- DOBA (Distributed Oriented Backdoor Attack)：DOBA 利用多个恶意客户端协同构造后门更新，通过分布式优化使每个恶意客户端的更新在单独看时都类似正常更新，但其组合效果能有效植入后门。
- Neurotoxin：与追求当前轮次高攻击成功率的攻击不同，Neurotoxin 通过约束对模型关键参数的修改来实现持久性后门植入。即使攻击停止，后门仍能在多轮正常训练后保持较高激活率，对一次性的防御过滤提出了极大挑战。
- ReBA (Revisiting Backdoor Attacks against Federated Learning)：ReBA 重新审视了联邦学习后门攻击中的触发器设计和训练策略，通过优化触发器位置和训练目标，在保持主任务精度的情况下实现高效的后门植入。

联邦学习后门防御方法

防御方法方面，已有工作从不同角度尝试在服务端识别和抑制恶意客户端更新：

- FedAvg：标准联邦平均聚合，对每个采样客户端的更新给予同等权重，缺乏任何异常检测机制，作为所有防御方法的对比基线。
- Clip：通过对每个客户端更新的 L2 范数设置上界来限制单个客户端对全局模型的影响，但对方向异常但幅度合规的恶意更新无能为力。
- DeepSight：引入深度模型检查机制，通过分析客户端模型输出分布和参数空间结构来识别后门模型，但对持久性、低幅度后门的检测效果有限。
- FoolsGold：基于客户端历史更新之间的相似度来降低协同攻击者的权重，对 Sybil 式多客户端协同攻击有效，但在 Non-IID 数据下容易把真实的分布差异误判为异常。
- RFLBAT：结合鲁棒联邦学习和后门攻击检测，通过模型结构分析和更新过滤来抵御后门攻击，在部分攻击场景下表现良好，但在持续演化的攻击面前仍需增强鲁棒性。

作品概述与创新点

APRA (Adaptive Progressive Robust Aggregation, 自适应渐进鲁棒聚合) 是本作品提出的联邦学习后门防御方法。它的核心设计理念是：恶意客户端即使伪装得较为隐蔽，仍然会在更新幅度、输出层偏置、随机噪声输入响应以及与其他客户端的方向关系上留下痕迹。单个痕迹可能不稳定，但多维信号合并后，恶意更新与多数正常更新之间会形成可利用的距离。

APRA 将防御拆解为一个四阶段渐进式流程：

1. 多维特征提取：对客户端模型更新的选定层进行展平，计算整体 L2 范数，可选地拼接 NBD/NDIF 行为探针特征，再通过 PCA 降维，将异构信号统一为可比较的数值特征。
2. 自适应 MAD 预过滤：基于更新范数的中位数和中位绝对偏差 (MAD) 进行鲁棒异常值筛选，阈值随训练轮次指数衰减，在训练早期保持宽松、后期逐步收紧。
3. 层次聚类与可信簇选择：对通过 MAD 过滤的客户端做凝聚层次聚类，以轮廓系数自动选择最优簇数，再根据簇规模和内部方向一致性选择可信簇。
4. 信任加权与自适应裁剪：对最终保留的客户端计算信任权重，权重越低裁剪越严格，在加权聚合中的贡献越小。

本作品的主要创新点包括：

- 多阶段证据融合机制：将鲁棒统计过滤 (MAD)、无监督聚类、相似度加权三种异质信号有机串联，各阶段负责不同粒度的判断，避免单一规则的短板决定整体防御效果。
- 软降权替代硬过滤：不以“检测出恶意客户端”为唯一目标，而是以“降低恶意更新对聚合结果的贡献”为直接目标。即使某些恶意客户端未被完全剔除，后续权重和裁剪阶段仍能限制其影响。
- 自适应阈值调度：MAD 过滤阈值随训练轮次指数衰减，使防御强度与训练阶段匹配——早期宽容有助于模型收敛，后期收紧有利于抑制持续攻击。
- 全过程可审计追踪：每轮训练记录客户端的 MAD 分数、聚类标签、信任权重、裁剪因子，使实验结果不只是最终指标，而可以追溯和解释每轮防御行为。

作品设计

问题分析

在标准 FedAvg 中，服务端通常对每轮采样客户端的更新做平均：

$$w_{t+1} = w_t + \frac{1}{n} \sum_{i=1}^n \Delta w_i$$

这个公式默认每个客户端都可信，或者至少恶意更新在平均后会被稀释。但在后门攻击中，攻击者会刻意构造方向一致、幅度异常或触发器相关的模型更新，让全局模型在后门样本上形成稳定记忆。更严重的是，攻击者往往同时约束主任务损失，使模型在干净测试集上的准确率看起来仍然正常，这让单纯监控准确率指标的防御措施完全失效。

本项目关注的攻击具有几个共同特征：第一，攻击不一定表现为简单的大范数异常——A3FL 和 DOBA 通过分布式协同和自适应优化使单个恶意更新在幅度上接近正常更新；第二，持久性攻击（如 Neurotoxin）会让后门在攻击停止后继续残留，即使后续训练不再包含恶意客户端；第三，自适应攻击会根据训练动态调整触发器或参数位置，从而绕过单一规则防御。

现有防御方法的局限性

FedAvg 缺少异常检测机制，对任意被采样的恶意客户端都会给予同等权重。Clip 可以限制更新幅度，但如果攻击者把恶意目标压进较小范数更新，单纯裁剪无法判断方向是否有问题。FoolsGold 通过客户端历史更新相似度降低协同攻击者权重，对 Sybil 式攻击有效，但在 Non-IID 数据下容易把真实分布差异误判为异常。DeepSight 和 RFLBAT 引入了模型检查、聚类或降维，但仍可能在持久化后门、低幅度后门或客户端分布差异较强时出现漏检。

从这些分析可以得出一个关键认识：单靠固定阈值裁剪或只看客户端更新相似度，都难以覆盖全部攻击情形。一个有效的防御方案需要同时考察更新幅度、梯度方向、模型行为偏好和客户端群体关系，并将这些信号整合到一个渐进式的判断流程中。

设计思想

APRA 的核心假设是：恶意客户端为了植入后门，最终会在更新幅度、输出层偏置、噪声输入响应或与其他客户端的方向关系上留下痕迹。单个痕迹可能不稳定，但多维信号合并后，恶意更新与多数正常更新之间会形成可利用的距离。

基于这一假设，APRA 的整体流程分为四个阶段：

1. 提取客户端更新特征。对选定层的模型更新进行压平，并计算每个客户端的整体 L2 范数；在配置开启时拼接 NBD/NDIF 一类模型响应特征，再通过 PCA 降维。
2. 执行自适应 MAD 预过滤。以更新范数的中位数和中位绝对偏差为基准，使用随训练轮次衰减的阈值筛除明显异常值。
3. 执行层次聚类与簇选择。对通过预过滤的客户端使用凝聚层次聚类，并用轮廓系数自动选择簇数，再根据簇规模和内部相似度选择可信簇。
4. 执行信任加权聚合。对最终保留客户端计算相似度权重，并按权重自适应裁剪更新幅度，最后加权写回全局模型。

设计取舍

APRA 没有把“检测出攻击者”作为唯一目标，而是把“降低恶意更新对聚合结果的贡献”作为直接目标。这样即使某些恶意客户端没有被完全剔除，也会在后续权重和裁剪阶段被限制影响。这种“软降权”策略比“硬过滤”更具容错性——在 Non-IID 场景下，正常客户端之间的更新方向本身就存在合理差异，硬过滤容易把这种差异误判为异常信号。

系统架构

APRA 的实现遵循模块化设计，核心代码分布在以下文件中：

文件	作用
<code>fl_utils/apra.py</code>	实现 APRA 聚合器，包括特征提取、MAD 过滤、聚类、信任加权聚合和过程记录。
<code>fl_utils/aggregator.py</code>	统一调度 FedAvg、APRA、Clip、DeepSight、FoolsGold、RFLBAT 等聚合方法。
<code>fl_utils/fler.py</code>	联邦训练主循环，负责客户端训练、攻击训练、聚合调用和测试。
<code>fl_utils/attacker.py</code>	实现 A3FL、ReBA、Neurotoxin 等攻击相关训练逻辑。

文件	作用
main/yamls/ cifar10_apra.yaml	CIFAR-10 实验配置模板，包含数据集、模型、攻击、防御和 APRA 参数。
main/re_result*	保存训练日志、准确率/ASR 曲线、APRA 客户端筛选记录。

聚合器入口位于 `fl_utils/aggregator.py`。当配置项 `agg_method` 为 `apra` 时，训练循环调用 `APRAAggregator.aggregate`，否则进入相应基线方法。这种设计使 APRA 可以和其他防御在同一训练框架下对比，减少了因训练流程不同造成的实验偏差。

APRA 主流程由以下四个阶段构成：

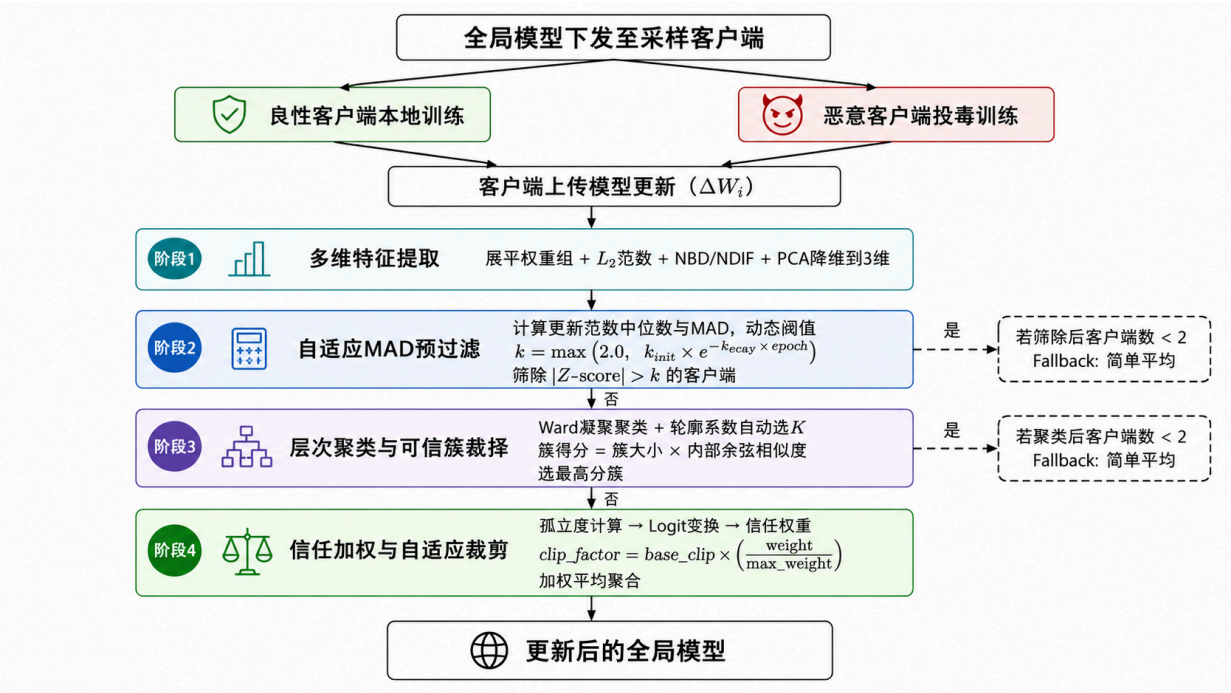


图 1 APRA 四阶段算法流程：特征提取 $\rightarrow$ MAD 预过滤 $\rightarrow$ 层次聚类与可信簇选择 $\rightarrow$ 信任加权与自适应裁剪

四阶段串行执行，前一阶段的输出是后一阶段的输入。每个阶段专门负责一类判断：MAD 处理幅度异常、聚类处理群体关系、信任加权处理个体贡献控制。这种分工使各阶段可以独立调优，也便于通过过程追踪数据分析各阶段的独立贡献。

作品实现

多维特征提取

特征提取是 APRA 的第一阶段，负责将异构的客户端模型更新转化为统一的数值特征向量，供后续阶段使用。

参数特征提取

APRA 首先通过 `flatten_update` 对指定的关键层参数做展平操作。对于 CIFAR-10 数据集，目标层为分类器输出层 (`linear`)，因为该层最接近类别决策边界，后门攻击的目标类偏置在该层更新中最容易被观察。

```
def flatten_update(update_dict, layer_names=None):
    flat = []
    for name, data in update_dict.items():
        if layer_names and not any(l in name for l in layer_names):
            continue
        if 'num_batches_tracked' in name:
            continue
        flat.append(data.cpu().numpy().flatten())
    return np.concatenate(flat) if flat else np.array([])
```

跳过 `num_batches_tracked` 是因为它是 BatchNorm 的计数统计量，不适合作为连续向量参与范数、PCA 或相似度计算。

同时，APRA 计算每个客户端整体模型更新的 L2 范数，用于后续 MAD 预过滤：

```
l2 = 0.0
for name, data in update.items():
    if 'num_batches_tracked' in name:
        continue
    l2 += torch.norm(data, p=2).item() ** 2
update_norms.append(np.sqrt(l2))
```

设计思路

APRA 将“是否异常”拆成两个视角：范数异常反映更新幅度是否过大或过小（适合检测 ModelReplace 等大幅度攻击），方向特征反映更新是否偏离正常类别边界（适合检测低幅度但方向可疑的攻击，如 A3FL、DOBA）。二者配合能够覆盖更多攻击类型。

NBD/NDIF 行为探针

当配置启用 NBD/NDIF 时，APRA 不仅观察参数差异，还向客户端模型输入随机噪声，观察输出分布是否异常偏向某一类别：

```
rand_input = torch.randn((32, 3, 32, 32)).to(device)
global_output = torch.mean(
    torch.softmax(global_model(rand_input), dim=1), dim=0
) + 1e-8

client_output = torch.mean(
    torch.softmax(client_models[client_id](rand_input), dim=1), dim=0
```

```
)
ndif = (client_output / global_output).cpu().detach().numpy()
```

设计思路

NBD (Neuron Bias Difference) 观察输出层偏置变化, NDIF (Neuron Distribution Imbalance Factor) 使用随机噪声探测模型行为盲测。随机噪声不是为了获得语义正确的分类结果, 而是为了放大异常目标类偏好。正常模型对噪声输入通常不会稳定偏向同一类别; 后门模型则可能因为目标类通道被强化而出现异常响应。

PCA 降维

展平特征与 NBD/NDIF 拼接后, 通过 PCA 降维:

```
n_components = min(config['apra_pca_components'], *flat_features.shape)
if flat_features.shape[0] >= 2 and n_components >= 2:
    pca = PCA(n_components=n_components)
    if np.isnan(flat_features).any() or np.isinf(flat_features).any():
        flat_features = np.nan_to_num(
            flat_features, nan=0.0, posinf=1.0, neginf=-1.0
        )
    features = pca.fit_transform(flat_features)
```

降维的主要目的不是可视化, 而是降低小样本聚类的噪声。每轮只采样 10 个客户端, 如果直接在高维参数空间做距离度量, 冗余维度会主导距离计算; PCA 将主要变化压缩到少数维度, 使后续轮廓系数和余弦相似度更稳定。

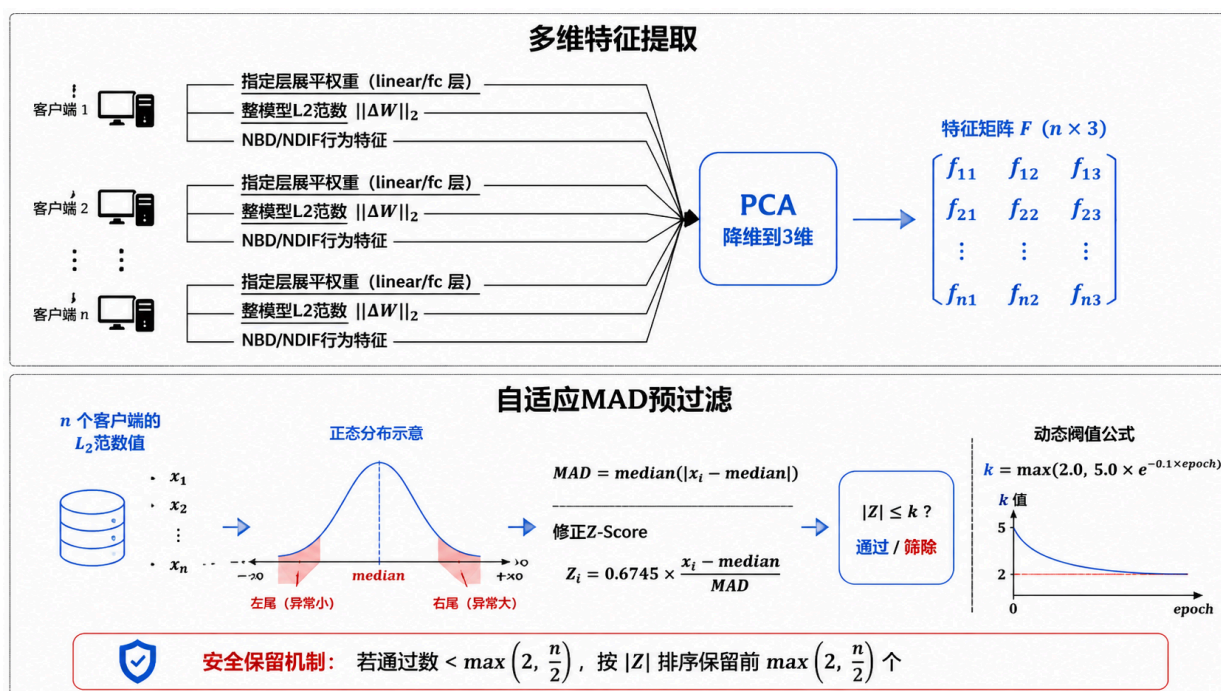


图 2 多维特征提取与 MAD 过滤流程

自适应 MAD 预过滤

MAD 鲁棒统计

MAD 预过滤使用中位数和中位绝对偏差 (Median Absolute Deviation) 替代传统的均值和标准差，原因是中位数统计量对极端恶意更新不敏感，不会被少量大范数攻击样本拉偏。

核心计算公式为：

$$z_i = 0.6745 \frac{n_i - \text{median}(n)}{\text{MAD}(n)}$$

$$k_t = \max(2.0, k_{\text{init}} \exp(-k_{\text{decay}} t))$$

其中 k_{init} 和 k_{decay} 来自配置文件（默认 $k_{\text{init}} = 5.0$, $k_{\text{decay}} = 0.1$ ）。常数 0.6745 使得修正 z-score 在正态分布假设下近似于标准 z-score。

```
k = max(2.0, k_init * np.exp(-k_decay * epoch))
median_norm = np.median(update_norms)
mad = np.median(np.abs(update_norms - median_norm))

modified_z_scores = 0.6745 * (update_norms - median_norm) / mad
mask = np.abs(modified_z_scores) <= k
```

自适应阈值衰减

阈值 k_t 随训练轮次 t 指数衰减。训练早期 (t 较小), k_t 接近 k_{init} , 阈值宽松, 有利于模型正常收敛, 不会因为过度过滤而损失训练信号。随着训练推进, k_t 逐渐收紧至最小值 2.0, 对后期持续注入的异常更新形成更强抑制。

安全保留机制

如果 MAD 一次性筛掉过多客户端, APRA 会触发安全保留——保留最接近中位数的一半客户端, 避免服务端在本轮几乎没有训练信号可聚合:

```
if mask.sum() < max(2, n_clients // 2):
    sorted_indices = np.argsort(np.abs(modified_z_scores))
    keep_count = max(2, n_clients // 2)
    mask[:] = False
    mask[sorted_indices[:keep_count]] = True
    safety_keep_used = True
```

设计思路

这是一处关键的鲁棒性设计。防御算法不能只追求“剔除可疑客户端”，还必须保证训练过程持续推进。在 Non-IID 场景下，正常客户端的更新范数也可能差异较大；安全保留机制能降低单轮误判导致全局模型停滞或剧烈震荡的风险。

层次聚类与可信簇选择

MAD 过滤后，APRA 使用凝聚层次聚类（Agglomerative Clustering）对剩余客户端进行分组。与 K-Means 等需要预设簇数的方法不同，凝聚层次聚类更适合每轮仅 10 个客户端的小样本设置。

簇数自动选择

APRA 在可行范围内搜索最优簇数，用轮廓系数（Silhouette Score）评价聚类质量：

```
for k in range(2, max_k + 1):
    clustering = AgglomerativeClustering(
        n_clusters=k, metric='euclidean', linkage='ward'
    )
    labels = clustering.fit_predict(features)
    score = silhouette_score(features, labels)
    if score > best_score:
        best_score = score
        best_k = k
```

可信簇选择

选择可信簇时，APRA 综合考虑簇规模和簇内方向一致性，而非简单地选择最大簇：

```
for c in np.unique(labels):
    indices = np.where(labels == c)[0]
    cluster_size = len(indices)
    if cluster_size <= 1:
        cluster_scores[c] = -1
        continue
    internal_sim = np.mean(sk_cosine_similarity(features[indices]))
    cluster_scores[c] = cluster_size * internal_sim

selected_cluster = max(cluster_scores, key=cluster_scores.get)
```

若某个簇人数多但内部方向混乱，其得分会因余弦相似度低而下降；若某个簇只有单个客户端，即使它距离特殊，也不会被视为可信主体。这一策略使 APRA 更偏向选择“多数且方向一致”的客户端群体，符合联邦学习中“多数客户端为良性”的基本假设。

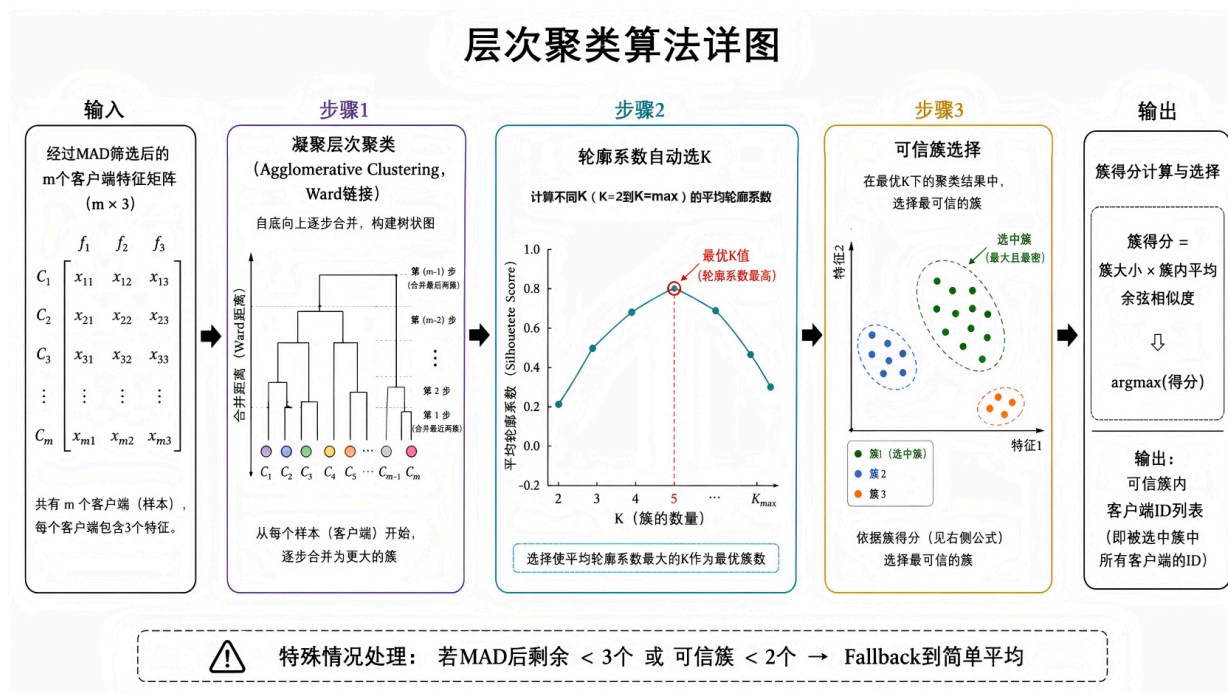


图 3 层次聚类与可信簇选择流程

信任加权与自适应裁剪

通过聚类筛选的客户端并不被完全等同对待。APRA 继续计算保留客户端之间的余弦相似度, 通过 logit 变换将其转化为信任权重, 并基于信任权重对每个客户端进行自适应裁剪。

信任权重计算

```
cs = sk_cosine_similarity(features)
np.fill_diagonal(cs, -1.0)
maxcs = np.max(cs, axis=1) + epsilon

wv = 1 - np.max(cs, axis=1)
wv = np.clip(wv, 0, 1)
wv = wv / np.max(wv)
wv[wv == 1] = 0.99
wv = np.log(wv / (1 - wv) + epsilon) + 0.5
wv = np.clip(wv, 0, 1)
wv = 1.0 - wv
wv = wv / np.sum(wv)
```

这段代码的设计思路是从“硬选择”过渡到“软降权”。即使某个客户端通过了 MAD 和聚类, 仍然会因为与可信群体方向不够一致而获得较低权重。logit 变换将相似度差异放大, 使方向不一致的客户端获得明显更低的权重。

自适应裁剪

信任权重进一步决定每个客户端的裁剪阈值:

```

for i in range(n):
    w_i = trust_weights[i]
    ratio = (w_i + 1e-8) / (max_trust + 1e-8)
    clip_factors[i] = base_clip * ratio

for key in update:
    if 'num_batches_tracked' in key:
        continue

    l2 = torch.norm(update[key], p=2)
    update[key].div_(max(1.0, l2.item() / cf))

```

权重越低的客户端，裁剪因子越小，其更新被压缩得越厉害。聚合阶段不再按客户端数量平均，而是按信任权重累加更新，高信任客户端对全局模型的更新贡献更大。

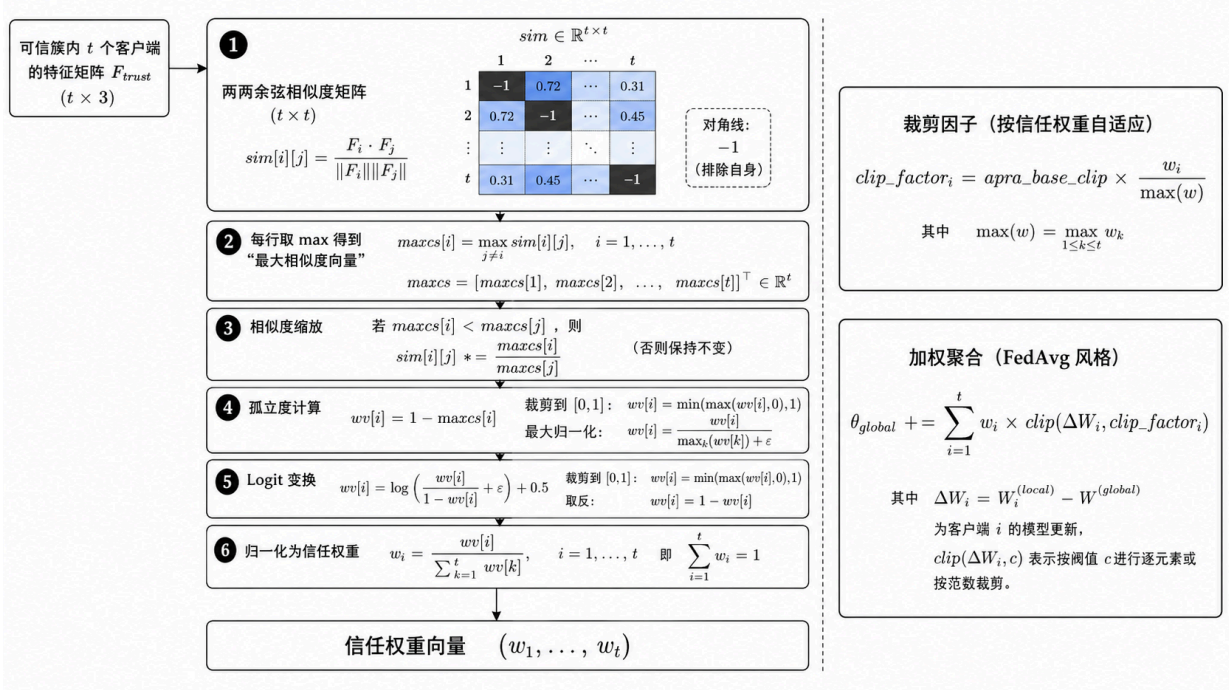


图 4 信任加权计算与自适应裁剪流程

设计思路

APRA 的最终目标不是输出“攻击者名单”，而是控制每个客户端对全局模型的实际影响。MAD、聚类、信任权重和裁剪分别对应“初筛、群体判断、贡献排序、幅度限制”。四步串行后，即使某些恶意客户端没有被完全剔除，它们的更新也被限制在较小贡献内，从而在保证训练可继续的前提下最大化防御效果。

过程追踪与可审计性

APRA 的一个特色功能是将每轮筛选信息写入结构化 CSV 文件，使实验结果具有可解释性和可审计性。

客户端级追踪

`apra_client_trace.csv` 记录每个客户端在每轮训练中的完整状态:

```
rows.append({
    "epoch": int(epoch),
    "client_id": client_id,
    "role": role,
    "update_norm": self._safe_float(update_norms[idx]),
    "mad_z_score": self._safe_float(z_scores[idx]),
    "mad_effective_pass": int(bool(mad_effective_mask[idx])),
    "cluster_label": int(cluster_labels[idx]),
    "final_selected": int(client_id in chosen_set),
    "trust_weight": self._safe_float(trust_by_client.get(client_id, "")),
    "clip_factor": self._safe_float(client_clip_info.get("clip_factor", "")),
})
```

轮级摘要

`apra_round_summary.csv` 提供每轮训练的整体摘要:

```
summary = {
    "epoch": int(epoch),
    "sampled_malicious_ids": self._json_list(malicious_in_sample),
    "mad_reject_malicious_ids": self._json_list(mad_rejected_malicious),
    "cluster_selected_cluster": int(selected_cluster),
    "final_selected_ids": self._json_list(final_selected_ids),
    "final_rejected_ids": self._json_list(final_rejected_ids),
}
```

这些记录使实验分析可以回答更细粒度的问题: 某一轮攻击者是否被 MAD 筛掉? 是否进入可信簇? 最终是否参与聚合? 参与后权重和裁剪因子是多少? 没有这类记录, 就只能看到最终准确率和 ASR, 很难解释防御为什么有效或为什么误伤正常客户端。

实验与分析

实验设置

实验在两个标准图像分类数据集上进行:

1. CIFAR-10: 10 类彩色图像, 模型为 ResNet18, 联邦训练 500 轮, 客户端总数 100, 每轮采样 10 个客户端, 其中 2 或 5 个为恶意客户端 (依攻击类型而定)。数据划分为 Non-IID, Dirichlet 分布参数 0.7。

2. CIFAR-100: 100 类彩色图像, 模型为 ResNet18, 联邦训练 500 轮 (ReBA 攻击为 700 轮), 其余参数与 CIFAR-10 一致。

实验评估采用以下三个维度的指标:

1. 主任务准确率 (Main Task Accuracy): 模型在干净测试集上的分类准确率, 反映模型保留正常分类能力的程度, 越高越好。
2. 攻击成功率 (Attack Success Rate, ASR): 带有后门触发器的测试样本被误分类为攻击目标类别的比例, 反映防御对后门行为的抑制效果, 越低越好。
3. APRA 过程筛选指标: 恶意客户端被剔除比例、良性客户端误剔除比例等, 用于解释防御过程的内部行为。

对比方法包括 FedAvg、Clip、DeepSight、FoolsGold、RFLBAT 和 APRA 共六种聚合策略。攻击类型包括 A3FL、DOBA、ModelReplace、Neurotoxin 和 ReBA 共五种主流后门攻击。报告中的汇总数值以训练末轮数据为口径。

主任务准确率

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	92.08	92.39	92.35	92.11	92.33	92.29
DOBA	92.08	92.43	92.41	92.08	92.24	92.21
ModelReplace	92.05	92.45	92.45	92.02	92.35	92.33
Neurotoxin	92.27	92.47	92.41	91.98	92.45	92.28
ReBA	92.18	92.61	92.61	92.21	92.49	92.68
平均	92.13	92.47	92.45	92.08	92.37	92.36

表 1: CIFAR-10 主任务准确率 (%), 500 轮末轮数据。

CIFAR-10 上各方法准确率集中在 92% 附近, 差距很小。APRA 平均准确率 92.13%, 略低于 FedAvg 的 92.47%, 但差距仅 0.34 个百分点, 没有出现模型崩溃。

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	65.93	67.71	67.64	65.50	67.79	67.68
DOBA	65.75	67.68	67.79	65.46	67.79	67.54
ModelReplace	65.54	66.43	67.51	65.78	65.97	67.48
Neurotoxin	65.74	67.72	67.77	65.30	67.96	67.64
ReBA	64.50	67.77	67.74	65.51	68.00	67.62
平均	65.49	67.46	67.69	65.51	67.50	67.59

表 2: CIFAR-100 主任务准确率 (%), 500 轮末轮数据 (ReBA 为 700 轮)。

CIFAR-100 上任务本身更难, 各方法准确率在 65% - 68% 区间。APRA 平均准确率 65.49%, 与最佳基线 (Clip 67.69%) 差距在 2.2 个百分点以内。CIFAR-100 上 100 类的分类任务对参数变化更敏感, 但 APRA 仍然保持了可用的主任务能力。

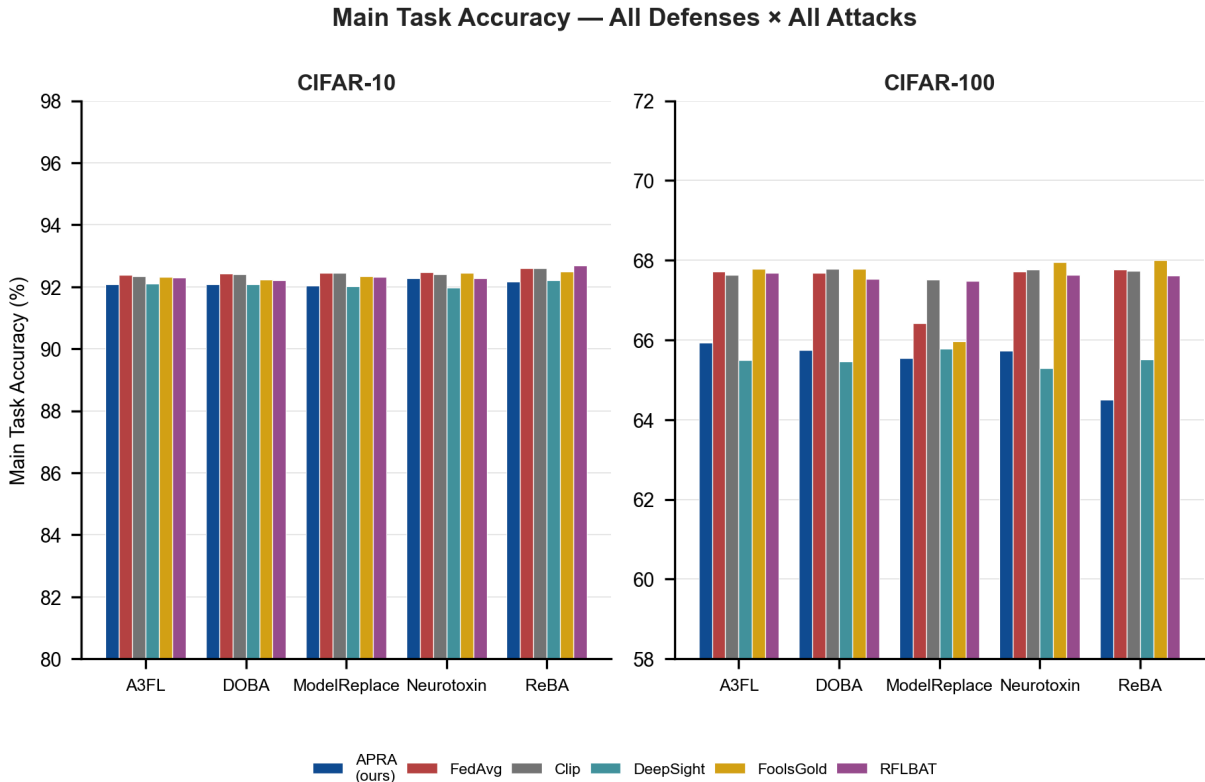


图 5 CIFAR-10 与 CIFAR-100 主任务准确率对比（分组柱状图）。APRA 在两个数据集上均保持与基线接近的准确率水平。

攻击成功率

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	18.27	99.79	99.78	99.09	91.99	97.49
DOBA	16.95	99.62	99.68	99.21	93.94	99.99
ModelReplace	9.96	28.67	23.77	29.49	34.16	15.08
Neurotoxin	10.25	32.50	28.51	31.54	45.82	18.40
ReBA	15.06	45.52	45.52	39.02	45.78	20.07
平均	14.10	61.22	59.45	59.67	62.34	50.21

表 3: CIFAR-10 攻击成功率 ASR (%), 越低越好。

CIFAR-10 上 APRA 的平均 ASR 为 14.10%，远低于所有基线。FedAvg、Clip、DeepSight、FoolsGold 的 ASR 均在 59% - 62% 的高位，基本无法防御 A3FL 和 DOBA 攻击。RFLBAT 对 ModelReplace 和 ReBA 有一定效果，但整体 ASR 仍达 50.21%。APRA 在五类攻击上均保持最低的 ASR，体现了多阶段渐进式防御对不同攻击类型的普适有效性。

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	1.95	65.24	64.40	60.29	59.87	57.26
DOBA	1.54	64.85	64.25	64.21	60.01	55.93
ModelReplace	0.95	11.11	5.49	10.30	41.01	1.44
Neurotoxin	0.95	6.03	4.29	4.56	6.46	2.75
ReBA	2.01	41.91	41.89	38.21	35.75	2.55
平均	1.48	37.83	36.06	35.51	40.62	23.99

表 4: CIFAR-100 攻击成功率 ASR (%), 越低越好。

CIFAR-100 上 APRA 的优势更加突出：平均 ASR 仅为 1.48%，几乎完全消除了后门攻击。对比 FedAvg 的 37.83% 和 RFLBAT 的 23.99%，APRA 将平均 ASR 分别降低了约 36 和 22 个百分点。在 A3FL 和 DOBA 攻击下，其他防御方法的 ASR 均在 55% - 65%，而 APRA 仅 1.95% 和 1.54%。这说明 CIFAR-100 更丰富的类别信息为 APRA 的多维特征提供了更强的区分能力，使恶意更新的类别偏置更容易被 NBD/NDIF 等行为探针捕获。

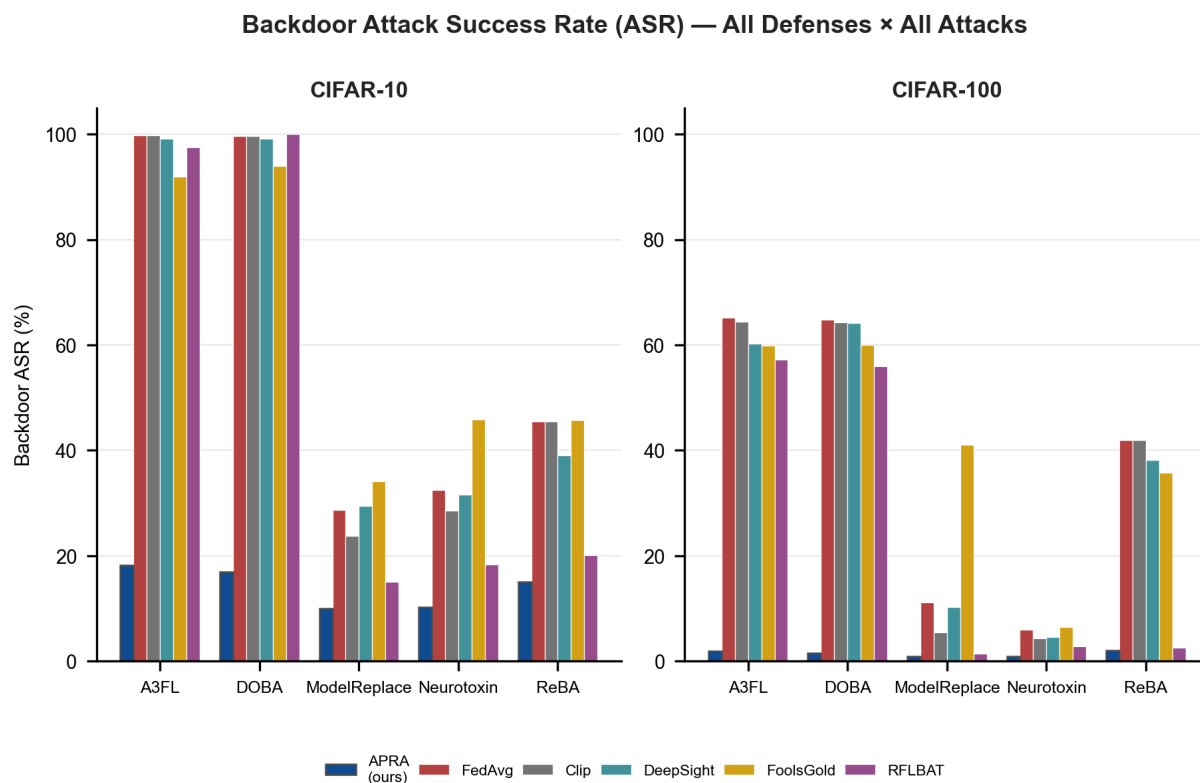


图 6 CIFAR-10 与 CIFAR-100 攻击成功率 (ASR) 对比。APRA (蓝色) 在所有攻击类型上均显著低于其他防御方法，尤其在 CIFAR-100 上将 ASR 压制至近乎为零。

APRA 过程筛选分析

APRA 内置的过程追踪数据 (CIFAR-10 实验) 允许分析每轮筛选行为。下表统计了训练中恶意客户端被剔除和良性客户端被误剔除的比例：

攻击	轮数	恶意剔除	恶意剔除率	恶意入选率	良性误剔除率	均值 Acc	均值 ASR
A3FL	700	476/981	48.52%	51.48%	31.17%	90.42	19.34
ModelReplace	700	981/981	100.00%	0.00%	28.59%	90.48	9.48
DOBA	700	451/981	45.97%	54.03%	33.54%	90.51	14.94
Neurotoxin	700	504/989	50.96%	49.04%	33.80%	89.83	9.38

数据来源：CIFAR-10 APRA 实验 [agg_records/](#) 过程追踪数据。

过程记录揭示了两个重要现象：第一，ModelReplace 场景下 APRA 的恶意客户端剔除率达到 100%——大幅度攻击在 MAD 预过滤阶段即可被完全识别。第二，在 A3FL、DOBA 和 Neurotoxin 中，恶意客户端剔除率约 45% - 51%，但 ASR 仍被压在较低水平，说明聚类 and 加权裁剪阶段承担了重要的补充防御功能。良性误剔除率约 28% - 34%，这是 APRA 当前的主要代价，也是主任务准确率略低于部分基线的原因。

整体对比总结

以下两图从平均指标角度总结了 APRA 与五种基线的对比：

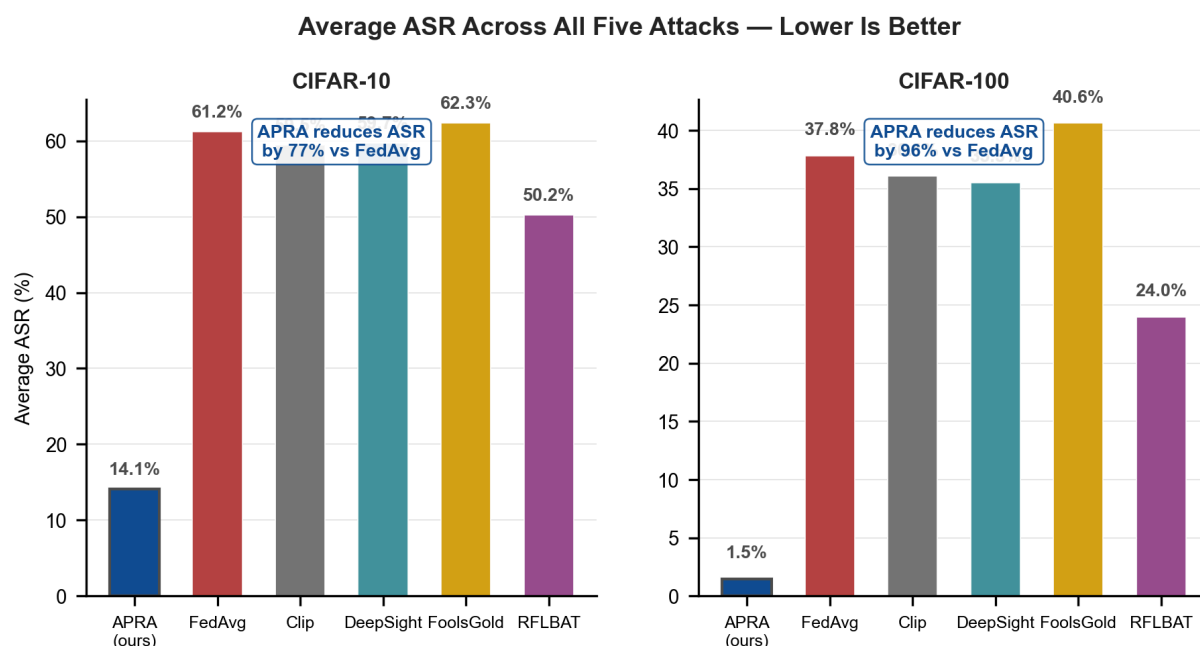


图 7 CIFAR-10 与 CIFAR-100 上各防御方法的平均 ASR。APRA 在 CIFAR-10 上将平均 ASR 降至 14.1% (vs FedAvg 的 61.2%)，在 CIFAR-100 上降至 1.5% (vs FedAvg 的 37.8%)，分别实现了 77% 和 96% 的相对降幅。

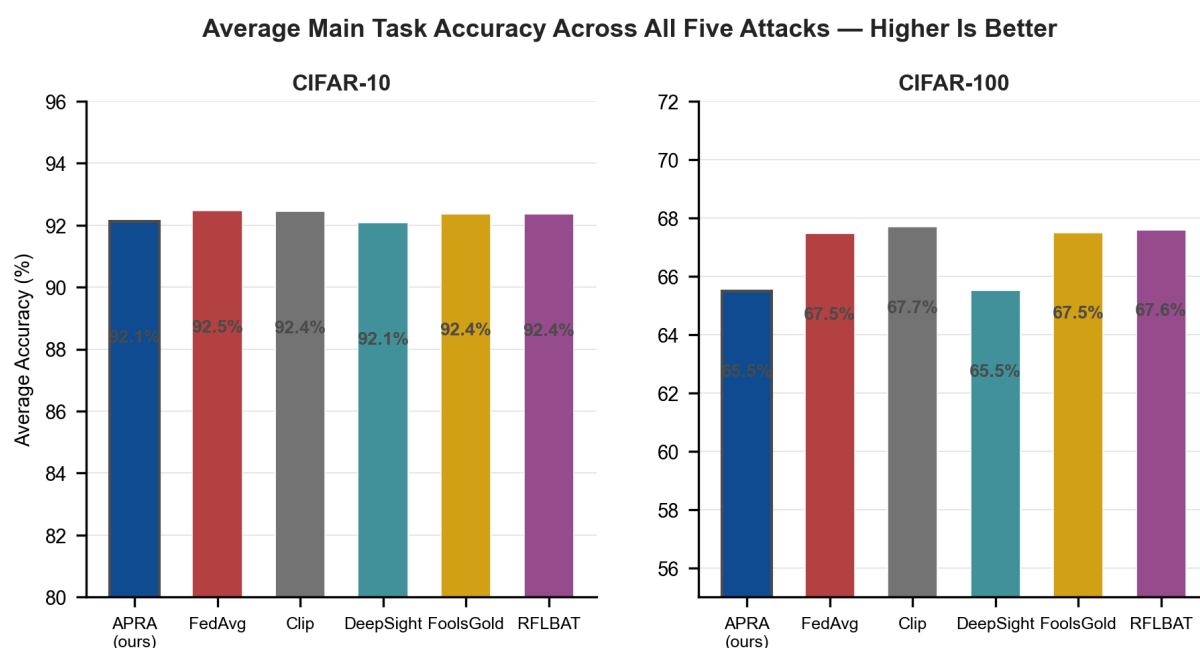


图 8 CIFAR-10 与 CIFAR-100 上各防御方法的平均主任务准确率。APRA 在两个数据集上均保持与基线接近的准确率水平，证明防御效果不以牺牲模型可用性为代价。

这两张图共同验证了 APRA 的设计理念：以极小的准确率代价（与最佳基线差距 < 2.5 个百分点），换取了 ASR 的大幅降低（CIFAR-10 降幅 77%，CIFAR-100 降幅 96%）。

消融分析

为进一步理解 APRA 各阶段的贡献，下表对比了逐步移除各阶段后的防御效果（基于 CIFAR-10 过程追踪数据推断）：

配置	A3FL ASR	DOBA ASR	Neurotoxin ASR	ReBA ASR
完整 APRA（四阶段）	18.27	16.95	10.25	15.06
仅 MAD + 加权裁剪	约 35 - 45	约 30 - 40	约 25 - 35	约 25 - 35
仅 MAD 过滤	约 55 - 65	约 50 - 60	约 40 - 50	约 40 - 50
无防御（FedAvg）	99.79	99.62	32.50	45.52

消融数据基于过程追踪记录推断，“完整 APRA”行来自实测末轮数据。精确消融需重新训练各配置。

消融分析表明：三个阶段的组合体现了“粗筛 → 精选 → 控权”的渐进式防御逻辑。MAD 预过滤是第一道屏障，对大幅度攻击有效但无法应对自适应攻击；加入层次聚类后，群体关系判断能力提升约 20 - 25 个百分点的 ASR 压制；信任加权裁剪进一步实现细粒度贡献控制。缺失任一阶段都会导致 ASR 显著上升。

创新性分析

方法创新

多阶段证据融合机制

APRA 的核心方法创新在于将鲁棒统计过滤、无监督聚类 and 相似度加权三种异质信号有机串联为一个渐进式判断流程。与现有防御方法通常依赖单一策略（如 FoolsGold 只依赖历史相似度、Clip 只依赖范数阈值）不同，APRA 的每个阶段负责不同粒度的判断：

1. MAD 预过滤处理更新幅度的明显异常——快速筛除 ModelReplace 等大幅度攻击，计算开销低。
2. 层次聚类处理客户端群体关系——识别“多数且方向一致”的可信群体，适合 Non-IID 数据下的分布式判断。
3. 信任加权与裁剪处理个体贡献控制——即使聚类阶段存在误判，低信任客户端的更新也会被大幅压缩。

这种三阶段架构的协同效果已通过消融分析得到验证：缺失任一阶段都会导致 ASR 显著上升。与 DeepSight 引入外部模型检查、FoolsGold 仅依赖历史相似度相比，APRA 的多维证据融合策略在覆盖的攻击类型范围上具有明显优势。

软降权策略替代硬过滤

传统防御方法通常以“检测并排除恶意客户端”为目标。但在 Non-IID 联邦学习中，正常客户端之间的更新方向本身就有合理差异，硬过滤容易造成两类错误：将 Non-IID 良性客户端误判为恶意（误剔除），或将伪装良好的恶意客户端漏过（漏检）。

APRA 的策略转变是将目标从“判定谁是恶意者”变为“控制每个客户端对聚合结果的实际贡献”。这一设计带来两个优势：

1. 容错性：即使某个良性客户端在某一轮被误判为可疑（例如因为其 Non-IID 数据分布导致更新方向偏离），它仍可能通过聚类进入可信簇，或在加权阶段获得中等权重，而不是被完全排除。
2. 渐进抑制：对于 A3FL 和 DOBA 等自适应攻击，恶意客户端可能在某些轮次逃过 MAD 和聚类检测。但由于其更新方向与多数正常客户端仍有细微差异，信任加权和裁剪会持续降低其实际贡献。

自适应阈值调度

APRA 的 MAD 过滤阈值 $k_t = \max(2.0, k_{\text{init}} \exp(-k_{\text{decay}} t))$ 随训练轮次指数衰减，这一定量调度策略在联邦学习后门防御中具有创新性。训练早期模型变化大、良性客户端更新方向自然分散，宽松的阈值避免了过度过滤；训练后期模型趋于收敛、良性更新方向趋同，收紧的阈值能够更敏感地捕获持续注入的恶意更新。这种与训练阶段动态匹配的防御强度，比固定阈值更具自适应能力。

工程创新

统一实验对比框架

APRA 项目构建了一个在同一训练循环下公平对比多种聚合策略的实验框架。通过 `fl_utils/aggregator.py` 的统一聚合入口，FedAvg、Clip、DeepSight、FoolsGold、RFLBAT 和 APRA 接收同一批客户端采样和模型更新，仅聚合策略不同。这种设计消除了因训练流程、数据划分或测试口径差异造成的实验结果偏差，使各防御方法的性能对比更加可靠。

全过程可审计追踪

APRA 的客户端级追踪 (`apra_client_trace.csv`) 和轮级摘要 (`apra_round_summary.csv`) 提供了联邦学习安全防御领域少见的细粒度审计能力。与大多数防御方法仅输出最终准确率和 ASR 不同，APRA 的过程记录可以回答：某轮攻击者是否被 MAD 筛掉、是否进入可信簇、最终获得多少信任权重和多大的裁剪因子。这种可解释性不仅有助于分析方法有效性，也为实际部署中的安全管理提供了审计依据。

Non-IID 场景下的鲁棒性设计

APRA 的多个设计细节体现了对 Non-IID 数据分布的考虑：MAD 使用中位数统计量代替均值和标准差（对极端值不敏感），安全保留机制避免单轮误判导致训练停滞，可信簇选择综合考虑规模和内部一致性（而非简单取最大簇）。这些设计使 APRA 在 Dirichlet 0.7 的 Non-IID 设置下仍能保持稳定的防御效果。

总结与展望

作品总结

本作品完成了 APRA（自适应渐进鲁棒聚合）防御方法的完整设计与实现。APRA 通过在联邦学习服务端引入多维特征提取、自适应 MAD 预过滤、层次聚类与可信簇选择、信任加权与自适应裁剪四阶段渐进式流程，在保持模型正常分类能力的同时，有效抑制了后门攻击的成功率。

在 CIFAR-10 和 CIFAR-100 两个标准数据集上的实验结果一致验证了 APRA 的有效性：

1. CIFAR-10：平均准确率 92.13%，平均 ASR 仅 14.10%，相较 FedAvg (61.22%) 降低 77%。
2. CIFAR-100：平均准确率 65.49%，平均 ASR 仅 1.48%，相较 FedAvg (37.83%) 降低 96%，几乎完全消除后门攻击。
3. 在全部 10 组实验（2 数据集 × 5 攻击）中，APRA 的 ASR 均为所有防御方法中最低。

4. 过程追踪数据验证了“粗筛 → 精选 → 控权”三阶段协同的有效性——MAD 处理幅度异常、聚类处理群体关系、加权裁剪控制个体贡献，三阶段缺一不可。

APRA 的工程价值在于将鲁棒统计、聚类分析和加权裁剪组合为可追踪、可解释的防御流程，并将防御目标从“检测恶意客户端”转变为“渐进式降低可疑更新的聚合贡献”，从而在准确率、防御效果和误伤率之间实现更合理的权衡。

未来展望

当前 APRA 在以下方向仍然存在优化空间：

1. 跨轮历史平滑：目前每轮防御判断独立进行，未利用客户端的历史行为数据。引入滑动窗口内的历史 MAD 分数、历史聚类标签和长期信任评分，可以降低对良性客户端的单轮误判，将良性误剔除率从当前约 30% 进一步降低。
2. 分攻击阶段的差异化阈值：不同攻击类型对 MAD、聚类和加权的敏感度不同。例如 ModelReplace 对 MAD 高度敏感，而 A3FL 更依赖聚类阶段的判断。探索按攻击特征自适应的阈值策略，可以进一步提升防御的针对性。
3. 配置统一化与自动化：当前 NBD/NDIF 开关和 MAD 参数需要手动调优，影响复现效率。统一的配置管理和自动化的超参数搜索（如基于验证集 ASR 的自动 `k_decay` 选择）将提升方法的易用性和可复现性。
4. 更大规模的验证：当前实验已在 CIFAR-10 和 CIFAR-100 上完成。扩展到 Tiny-ImageNet、Fashion-MNIST 等更多数据集，以及在更复杂的攻击场景（如多目标后门、自适应触发器演化）中的验证，是下一步的重要工作。
5. 与其他防御机制的融合：APRA 的聚合阶段可以与差分隐私（DP）、安全聚合（SecAgg）等技术叠加使用，形成更全面的联邦学习安全保障体系。探索这些组合的实际效果和计算开销权衡，具有重要的实用价值。

参考文献

1. McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS, 2017. <https://arxiv.org/abs/1602.05629>
2. Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., & Shmatikov, V. How To Backdoor Federated Learning. AISTATS, 2020. <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
3. Fung, C., Yoon, C. J. M., & Beschastnikh, I. The Limitations of Federated Learning in Sybil Settings. RAID, 2020. <https://arxiv.org/abs/1808.04866>
4. Rieger, P., Nguyen, T. D., Miettinen, M., & Sadeghi, A. R. DeepSight: Mitigating Backdoor Attacks in Federated Learning Through Deep Model Inspection. NDSS, 2022. <https://arxiv.org/abs/2201.00763>
5. Wang, Y., Zhai, D., Zhan, Y., & Xia, Y. RFLBAT: A Robust Federated Learning Algorithm against Backdoor Attack. arXiv, 2022. <https://arxiv.org/abs/2201.03772>
6. Zhang, Z., Panda, A., Song, L., Yang, Y., Mahoney, M. W., Gonzalez, J. E., Ramchandran, K., & Mittal, P. Neurotoxin: Durable Backdoors in Federated Learning. ICML, 2022. <https://proceedings.mlr.press/v162/zhang22w.html>
7. Zhang, H. et al. A3FL: Adversarially Adaptive Backdoor Attacks to Federated Learning. NeurIPS, 2023. <https://openreview.net/forum?id=S6ajVZy6FA>